Lab 2A: Build Your First Skill

Duration: 35 min

After: Skills & Advanced CLAUDE.md lecture (Day 2, Block 1)

Prerequisites: Day 1 labs (business-dashboard with CLAUDE.md) - recovery prompt below if you fell behind

Objective

Create a custom skill with current SKILL.md frontmatter (`name`, `description`, `disable-model-invocation`, `allowed-tools`), write a "pushy" description that triggers reliably, and test activation systematically.

Deliverable (toolkit chain 5 of 12)

A working **code-review skill** at `.claude/skills/code-review/` - toolkit artifact #2 (after CLAUDE.md).

> ***START MARKER:*** *In* `business-dashboard`*,* `claude` *running,* `/clear` *done.*

Fell behind on Day 1? Run:

```
Create a Node.js Express project called "Business Dashboard" with: package.json (express
dependency), server.js with a basic Express server on port 3100, src/routes/tasks.js with
GET/POST /api/tasks endpoints (in-memory array), a CLAUDE.md with project description, tech
stack, and coding standards (async/await, guard clauses, parameterized queries), .gitignore,
and an initial git commit.
```

Exercise 1: Analyze Before You Build (5 min)

```
What are the 3 most common code review checks I should run on this project before
committing? Be specific to this codebase.
```

Write the three checks down - they go into your skill.

> ***6-Step Framework:*** *Name -> Trigger -> Outcome -> Dependencies -> Steps -> Edge Cases. Define "done" BEFORE writing instructions.*

Exercise 2: Create the Skill (10 min)

Step 1 - create it. Note the frontmatter fields - this is the current July 2026 format:

```
Create a code review skill at .claude/skills/code-review/SKILL.md with EXACTLY this YAML
frontmatter structure:

---
name: code-review
description: Reviews code for security, quality, and style issues. Use when the user says
"review this code", "check this file", "code review", "audit this", "look for issues",
"quality check", "security check", "find bugs", or "QA check". Do NOT use for writing new
code, refactoring, or documentation.
disable-model-invocation: false
allowed-tools: Read, Grep, Glob
---

Body workflow:
1. Read the specified files
2. Check security: hardcoded secrets, SQL injection, unsanitized input
3. Check quality: functions over 50 lines, missing error handling, unused imports
4. Check style: naming conventions, formatting consistency
5. Include these project-specific checks: [paste your 3 checks from Exercise 1]

Output format: [RED] Critical (security) / [YELLOW] Warning (quality) / [BLUE] Info (style)

Add this anti-hallucination rule: "If no issues are found, explicitly state 'No issues
found' - do not invent problems."
```

Step 2 - inspect:

```
!cat .claude/skills/code-review/SKILL.md
```

Expected output marker: frontmatter contains all four fields: `name`, `description`, `disable-model-invocation`, `allowed-tools`.

Check the description: 7+ trigger phrases, a negative boundary, written for the MODEL ("when should I fire?"), not for humans. This is the #1 skill failure point.

> ***Frontmatter cheat:*** `allowed-tools: Read, Grep, Glob` *restricts the skill to read-only tools.* `disable-model-invocation: true` *would make it manual-only (*`/code-review`*) - right for side-effect skills like /deploy; wrong for this one, where you WANT auto-triggering.*

Exercise 3: Test Activation (10 min)

Skill activation is probabilistic - the description is the trigger. Reload first:

```
/clear
```

#	Prompt	Should fire?
1	`Review the code in src/routes/tasks.js`	Yes
2	`Check this file for issues: @server.js`	Yes
3	`Can you do a quality check on @src/routes/`	Yes
4	`Add a new API endpoint for user profiles`	**No**
5	`Audit the dashboard code`	Yes

Expected output marker (tests 1,2,3,5): the [RED]/[YELLOW]/[BLUE] format from your skill. **Test 4:** Claude writes code - no review report. If test 4 fires the skill, your description is too broad: sharpen the negative boundary.

Check the skill inventory and token cost:

```
/skills
```

Expected output marker: `code-review` listed under Project skills. (Tomorrow's tie-in: the July 2026 `/doctor` flags skills that never fire - unused-skill detection.)

Exercise 4: Add a Reference File (5 min)

```
Create a reference checklist at .claude/skills/code-review/references/checklist.md with
security checks (no API keys, input sanitization, no eval), quality checks (error handling
on async, no console.log in production, single responsibility), and style checks (camelCase
variables, consistent imports). Use checkbox format.

Then update SKILL.md to add this instruction at the top of the workflow: "Before beginning
the review, read references/checklist.md to ensure all project-specific checks are covered."
```

Test:

```
Review @src/routes/tasks.js against our checklist
```

Expected output marker: the review explicitly references checklist items.

> ***One-level-deep rule:*** *references must not link to other references - Claude truncates nested reads. Target 1,500-2,000 words for the SKILL.md body; push bulk into references/ (progressive disclosure - you pay tokens only when it fires).*

Exercise 5: A Manual-Only Skill (5 min)

Build a side-effect skill that Claude must NOT auto-trigger:

```
Create a skill at .claude/skills/ship-checklist/SKILL.md with frontmatter:
name: ship-checklist
description: Pre-release checklist for this project. Runs tests, checks for console.log,
verifies version field, checks .env.example exists.
disable-model-invocation: true
allowed-tools: Read, Grep, Glob, Bash
Body: run each check and report PASS/FAIL with a final SHIP / DO NOT SHIP verdict.
```

```
/clear
```

Test that natural language does NOT trigger it: `Is this project ready to release?` - Claude may answer, but should not run the checklist skill automatically. Then invoke manually:

```
/ship-checklist
```

Expected output marker: the checklist runs only on explicit invocation. That's `disable-model-invocation: true` doing its job.

FINISH MARKER - Success Checklist

- [ ] `.claude/skills/code-review/SKILL.md` with all four frontmatter fields
- [ ] 7+ trigger phrases + negative boundary in description
- [ ] `allowed-tools` restricts to Read, Grep, Glob
- [ ] 4 of 5 activation tests behave correctly (incl. the negative test)
- [ ] references/checklist.md wired in and cited in output
- [ ] ship-checklist fires ONLY via /ship-checklist
- [ ] `/skills` shows both skills

If Stuck

Problem	Recovery
Skill never fires	Description too timid - add trigger phrases; be pushy
Skill fires on everything	Add "Do NOT use for..." boundaries
Skill file in the wrong place	Say "in the current project directory" - otherwise Claude may write to `~/.claude/` (global)
Changes not picked up	`/clear` reloads skills
No RED/YELLOW/BLUE format	The skill didn't activate - check `/skills` lists it, check the description

Debrief Questions

1. What is the description field actually for - and who is its audience?
2. When do you set `disable-model-invocation: true`?
3. Why restrict `allowed-tools` on a review skill - what failure does it prevent?